

从 count matrix 到 annotated matrix

count matrix == 计数矩阵

Annotated matrix == 带注释的矩阵

细胞、基因与表达计数

简单的计数矩阵：

cell_id	MS4A1	CD3D	NKG7	LYZ
cell_001	8	0	0	1
cell_002	0	12	1	0
cell_003	0	1	9	2

- 每一行代表一个细胞
- 每一列代表一个基因
- 数字代表某细胞对某基因的计数
 - 0表示未记录到，不能说是“未表达”

Observation、variable与两条轴

对于形状为 $n \times d$ 的矩阵

- 一行是一个observation
 - obs是细胞
 - n为obs数量
- 一列是一个variable（或feature）
 - var是基因
 - d为var数量

行列名称与metadata

若仅有无名称的 3 x 4数组，无法判断行名和列名，则需要一些描述信息，称作**metadata**，可以提供

- 每个细胞来自哪个样本、供体、实验条件
- 每个基因对应哪个gene ID、位于哪条chr

通常来说，.obs和.var就是metadata的2种形式，分别记录细胞和基因的其他信息

并且，metadata需要

- 沿obs轴与每个细胞逐一对应
- 沿var轴与每个基因逐一对齐

为什么需要逐一的数据容器

.X表达矩阵，.obs细胞信息表和.var基因信息表可以分别保存，但必须使用同一组身份和顺序。若数据顺序变化，相关信息的顺序也要变化

AnnData结构

准备数据

首先将前文的count matrix写成一个numpy数组，转成CSR稀疏矩阵

CSR, Compressed Sparse Row, 即压缩稀疏行格式

稠密矩阵：保存所有位置，包括大量 0

CSR 稀疏矩阵：主要保存非零值及其行列位置

```
counts_dense = np.array(  
    [  
        [8, 0, 0, 1],  
        [0, 12, 1, 0],  
        [0, 1, 9, 2],  
    ],  
    dtype=np.float32,  
)  
  
counts = csr_matrix(counts_dense)  
counts.shape # 输出(3,4)
```

创建AnnData对象

obs和var先仅提供索引，不提供其他metadata信息。索引分别给obs轴和var轴的每个位置命名

```
cell_names = ["cell_001", "cell_002", "cell_003"]  
gene_names = ["MS4A1", "CD3D", "NKG7", "LYZ"]  
  
obs = pd.DataFrame(index = cell_names)  
var = pd.DataFrame(index = gene_names)  
  
adata = ad.AnnData(X = counts, obs = obs, var = var)  
# 输出 n_obs x n_vars = 3 x 4
```

此时.obs和.var无额外的metadata，仅含有索引，知道这一行（或列）是谁

.X核心数据矩阵

.X是AnnData核心二维矩阵，第0轴与obs对齐，第1轴与var对齐，即：

```
adata.X.shape == (n_obs, n_vars) == (3, 4)
```

由于目前.X不大，可以使用以下代码来查看数据

```
adata.X.toarray()
```

.obs与.var的轴信息表

- .obs 是 observation 轴的信息表
 - 每行严格按照顺序对应 .X 的每一行
 - 通常含有很多信息，如
 - sample_id
 - condition等

```
adata.obs
```

	sample_id	condition
cell_001	sample_A	control
cell_002	sample_A	control
cell_003	sample_B	treated

- .var 是 variable 轴的信息表
 - 每行严格按照顺序对应 .X 的每一列
 - 通常含有很多信息，如
 - gene_id
 - feature_type等

adata.var

	gene_id	feature_type
MS4A1	gene_demo_001	Gene Expression
CD3D	gene_demo_002	Gene Expression
NKG7	gene_demo_003	Gene Expression
LYZ	gene_demo_004	Gene Expression

形状、轴名称和对象摘要

- .n_obs, .n_vars, .shape可以描述对象
- .obs_names, .var_names可以访问轴名称

```
print("shape:", adata.shape)
# shape: (3, 4)
print("n_obs:", adata.n_obs)
# n_obs: 3
print("n_vars:", adata.n_vars)
# n_vars: 4
print("obs_names:", adata.obs_names.tolist())
# obs_names: ['cell_001', 'cell_002', 'cell_003']
print("var_names:", adata.var_names.tolist())
# var_names: ['MS4A1', 'CD3D', 'NKG7', 'LYZ']
print(adata)
# AnnData object with n_obs × n_vars = 3 × 4
#   layers: None (.X)
```

添加并检查对齐的metadata

向.obs添加细胞级metadata

可以添加样本编号、实验条件和采集时间

```
adata.obs["sample_id"] = pd.Categorical(
    ["sample_A", "sample_A", "sample_B"]
)
adata.obs["condition"] = pd.Categorical(
    ["control", "control", "treated"]
)
adata.obs["collection_day"] = [0, 0, 7]

adata.obs
```

结果为

	sample_id	condition	collection_day
cell_001	sample_A	control	0
cell_002	sample_A	control	0
cell_003	sample_B	treated	7

向. var添加基因级metadata

```
adata.var["gene_id"] = [
    "ENSG_DEMO_001",
    "ENSG_DEMO_002",
    "ENSG_DEMO_003",
    "ENSG_DEMO_004",
]
adata.var["chromosome"] = pd.Categorical(["11", "11", "19", "12"])

adata.var
```

输出内容为

	gene_id	chromosome
MS4A1	ENSG_DEMO_001	11
CD3D	ENSG_DEMO_002	11
NKG7	ENSG_DEMO_003	19
LYZ	ENSG_DEMO_004	12

使用索引来对齐

如果外来的metadata表格顺序与adata.obs不同时，可以使用pandas通过细胞名称来对齐，而不是通过行的位置

```
external_obs = pd.DataFrame(  
    {"donor": ["donor_2", "donor_1", "donor_1"]},  
    index=["cell_003", "cell_001", "cell_002"],  
)  
  
adata.obs["donor"] = external_obs["donor"] # 通过赋值自动对齐
```

类别型、数值型和文本型字段

metadata列可以有不同的数据类型，可以通过 `adata.obs.dtypes` 来显示数据类型

- 类别型，即category
 - 重复出现的有序类别
 - 如condition
- 数值型，如int64
 - 连续或有序测量
 - 如collection_day
- 字符串，即str
 - 自由文本
 - 如donor

对齐检查

可以使用以下代码进行检查，若全显示True即正常

```
checks = {  
    "obs_rows_match": adata.obs.shape[0] == adata.n_obs,  
    "var_rows_match": adata.var.shape[0] == adata.n_vars,  
    "obs_index_match": adata.obs.index.equals(adata.obs_names),  
    "var_index_match": adata.var.index.equals(adata.var_names),  
    "obs_names_unique": adata.obs_names.is_unique,  
    "var_names_unique": adata.var_names.is_unique,  
}  
checks
```

选择细胞与基因

按位置切片

AnnData可以使用adata[obs_selection, var_selection]的二维形式，如

```
position_subset = adata[:2, :3]
position_subset.shape
```

代码截取了[0,2)行和[0,3)列，即前两个细胞和前三个基因的信息，输出为 (2, 3)

按名称切片

可以使用细胞名称.obs_names和基因名称.var_names来选择对象

```
name_subset = adata[
    ["cell_001", "cell_003"],
    ["MS4A1", "NKG7"],
]

name_subset.shape, name_subset.obs_names.tolist(), name_subset.var_names.tolist()
```

形状为 (2, 2)，细胞为 cell_001、cell_003，基因为 MS4A1、NKG7。

使用.obs条件进行布尔选择

由已有细胞级 metadata 构造布尔掩码，比如可以只选择condition为control的细胞，保留所有基因

```
control_mask = adata.obs["condition"] == "control"
control_subset = adata[control_mask, :]

control_mask.tolist(), control_subset.shape, control_subset.obs_names.tolist()
```

掩码为 [True, True, False]，结果形状为 (2, 4)，包含前两个细胞

切片时对齐内容同步变化

当从AnnData中选出部分细胞或基因时，AnnData不仅只切了.X，还同步切对应的.obs和.var，确保 .x, .obs, .var 的shape始终一致

.view

对AnnData切片时，通常返回的是一个**view**，作为原对象的一个视图，而不是立即复制所有底层数据，以节省内存，用于记录选择了哪些细胞和基因，通常仅用于查看或绘图

```
view_subset = adata[:2, :2]
view_subset.is_view # 返回True
```

.copy()

如果准备将子集作为**独立对象**进行数据处理，那么可以使用 `.copy()` 来将view转成持有实际数据的 AnnData对象

```
independent_subset = adata[:2, :2].copy()
independent_subset.is_view # 返回False
```

保存其他类型的数据

.layers：同一个二维空间的其他矩阵

`.layers` 是按键访问的矩阵集合，每个layer必须与 `.x` 具有相同形状

作用是在同一批细胞、同一批基因下，同时保存表达矩阵的不同版本，避免一个版本覆盖另一个版本。

例如，最开始的时候是原始技术矩阵；随后需要把X换成表达矩阵；再然后可能要规范化等处理。可以使用layers来同时保存不同状态的版本，主要靠的是 `.copy()`

.obs：observation级的多维矩阵

`.obs` 中每个值第一维必须等于`n_obs`，后续维度可以根据用途变化，其中的m为matrix

例如，在UMAP中，通常为 $n_{obs} \times 2$ 的矩阵，记录每个细胞的x和y坐标，可以看出两个细胞的差别不大

例如，在PCA中，可以用 `.obs["X_pca"]` 来表示每个细胞在 50 个PC上的坐标，形状为 $(n_{obs}, 50)$

```
adata.obs          ← 第一层:容器(字典,管"按名字存取")
|
├── "X_pca" → numpy.ndarray (27674, 30) ← 第二层:内容(多维矩阵)
└── "X_umap" → numpy.ndarray (27674, 2)
```

.var：variable级多维矩阵

与`.obs`对称，第一维同样等于`n_vars`

例如，在PCR中，可以用 `.vars["PCs"]` 来表示每个基因在 50 个PC上的权重，形状为 $(n_{vars}, 50)$

每个PC，即主成分实际上是基因的组合公式，如

$$PC1 = 0.8 \times CD3D - 0.4 \times MS4A1 + 0.2 \times NKG7 - 0.1 \times LYZ + \dots$$

其中里面的 (0.8, -0.4, 0.2, -0.1) 就存放在 `.varm["PCs"]`，可以用来找到每个PC的关键基因

.uns：非轴对齐信息

可以用来保存不属于细胞和基因的结构化信息，常常包含字典、列表、文本或其他全局信息，可以按键key访问，无固定行数约束

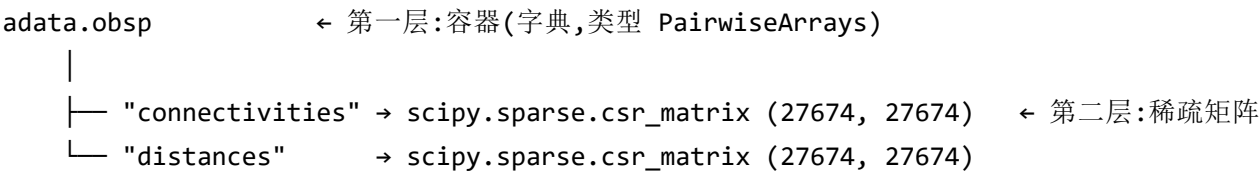
例如，聚类之后对每个类群标记颜色，这些既不属于细胞也不属于基因，是一些全局信息，属于.uns

.obsp：observation 两两关系矩阵

用来保存细胞之间的关系，shape为 $n_obs \times n_obs$ ，行和列都是细胞，矩阵里的 [i,j] 表示细胞i和细胞j之间的关系强度。其中p=pairwise，两两

如邻居图，即 `.obsp` 在scanpy的核心用途，可以创建两个矩阵

- 距离矩阵：范围不限于1，越小则距离越短，即关系越亲密
- 连接矩阵：从0到1，越大则关系越亲密，可以视作距离矩阵归一化之后反过来



存储位置与形状约束

位置	基本结构	对齐关系	概括
<code>.X</code>	二维矩阵	$n_obs \times n_vars$	表达矩阵
<code>.obs</code>	DataFrame	每行一个 observation	细胞信息
<code>.var</code>	DataFrame	每行一个 variable	基因信息
<code>.layers[key]</code>	二维矩阵	$n_obs \times n_vars$	数据版本
<code>.obsm[key]</code>	多维数组	第一维为 n_obs	细胞多维
<code>.varm[key]</code>	多维数组	第一维为 n_vars	基因多维

位置	基本结构	对齐关系	概括
.obsp[key]	方阵	n_obs × n_obs	细胞两两
.uns[key]	非固定结构	不要求逐轴对齐	补充信息

转换、保存与重新读取

to_df() 查看核心矩阵

to_df() 可以返回一个dataframe，以.obs_names为行索引，以.var_names为列名，不包括.obs等其他内容，仅为一个二维表达矩阵

```
expression_df = adata.to_df()
```

如果输入expression_df.obs会报错

.h5ad 与 AnnData 对象

adata是当前python进程中的AnnData对象，而.h5ad是将adata持久化到磁盘的文件格式，保存后即使关闭python也不会丢失adata

```
output_path = the_path
adata.write_h5ad(output_path)
```

同时，可以通过以下代码进行读取，恢复AnnData对象

```
restored = ad.read_h5ad(output_path)
```

不覆盖原始数据

原始的计数矩阵、metadata、源文件等应视为不可修改，学习和分析产生的AnnData应该写入明确派生路径，遵守文件安全原则

为 Scanpy 学习建立数据地图

常见结果可能存放的位置

未来可能见到的内容	常见位置	结构理由
每个细胞的标签或指标	.obs	每个 observation 一个值
每个基因的标签或指标	.var	每个 variable 一个值

未来可能见到的内容	常见位置	结构理由
不同版本的表达矩阵	.layers 或 .X	仍为细胞 × 基因
每个细胞的多维表示	.obsm	第一维为细胞数
细胞之间的成对关系	.obsp	细胞 × 细胞方阵
参数、颜色或全局说明	.uns	不逐细胞或逐基因对齐

检查一个陌生的AnnData对象

```
print(adata) # 看看摘要
print("shape:", adata.shape) # 看看shape
print("X type:", type(adata.X).__name__) # 看看.X矩阵类别
print("obs columns:", adata.obs.columns.tolist()) # 看看细胞
print("var columns:", adata.var.columns.tolist()) # 看看基因
print("layers:", list(adata.layers.keys()))
print("obsm:", list(adata.obsm.keys()))
print("varm:", list(adata.varm.keys()))
print("obsp:", list(adata.obsp.keys()))
print("uns:", list(adata.uns.keys()))
```